

Sistemas Operacionais 1

SINCRONIZAÇÃO DE PROCESSOS

Professor.: Dalton Matsuo Tavares

- Fundamentação teórica
- Problema da “seção crítica”
- Solução de Peterson
- Sincronização via Hardware
- Proteções via mutex (*Mutex Locks*)
- Semáforos
- Problemas clássicos de sincronização
- Monitores

- Fundamentação teórica
- Problema da “seção crítica”
- Solução de Peterson
- Sincronização via Hardware
- Proteções via mutex (*Mutex Locks*)
- Semáforos
- Problemas clássicos de sincronização
- Monitores

- Apresentar o conceito de sincronização de processos.
- Introduzir o problema de seção crítica, cujas soluções podem ser usadas para assegurar a consistência dos dados compartilhados
- Apresentar soluções de software e hardware para o problema da seção crítica
- Examinar vários problemas de sincronização clássicos
- Explorar várias ferramentas que são usadas para solucionar os problemas de sincronização de processo.

- Processos podem executar de forma concorrente
 - Podem ser interrompidos a qualquer momento, completando a execução parcialmente
- Acesso concorrente a dados compartilhados podem resultar em inconsistência de dados
- Manutenção de consistência requer mecanismos que assegurem a execução ordenada de processos cooperativos

- Ilustração do problema:
 - “Suponha que se queira oferecer uma solução para o problema do produtor-consumidor que preencha todos os *buffers*. Isso pode ser feito por meio de um contador inteiro (`counter`), o qual identifica a quantidade de *buffers* cheios. Inicialmente, `counter` é iniciado em 0. A variável é incrementada pelo produtor após produzir um novo dado para o *buffer* e é decrementado pelo consumidor após consumir o conteúdo de um *buffer*”.

```
while (true) {  
    /* produz um item em next_produced */  
    while (counter == BUFFER_SIZE);  
    /* Não faz nada - buffer cheio */  
  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

```
while (true) {  
    while (counter == 0);  
    /* Não faz nada - buffer vazio */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* Consome o item em next_consumed */  
}
```


- **counter++** poderia ser implementado como

- `register1 = counter`
 - `register1 = register1 + 1`
 - `counter = register1`

- **counter--** poderia ser implementado como

- `register2 = counter`
 - `register2 = register2 - 1`
 - `counter = register2`

Condição de Corrida (cont)

- Considere esta execução sobreposta com “counter = 5”:
- T₀: produtor executa `register1 = counter` {register₁ = 5}
- T₀: produtor executa `register1 = register1 + 1` {register₁ = 6}
- T₁: consumidor executa `register2 = counter` {register₂ = 5}
- T₁: consumidor executa `register2 = register2 - 1` {register₂ = 4}
- T₀: produtor executa `counter = register1` {counter = 6}
- T₁: consumidor executa `counter = register2` {counter = 4}

Problema da Seção Crítica

- Considere o sistema de n processos $\{p_0, p_1, \dots, p_{n-1}\}$
- Cada processo tem um segmento de código de **seção crítica**
 - Processo pode estar modificando variáveis comuns, atualizando tabelas, escrevendo arquivos etc
 - Quando um processo está na seção crítica, nenhum outro processo pode estar na seção crítica
- **Problema da seção crítica** consiste no projeto de um protocolo que resolva essa questão
- Cada processo deve pedir permissão para entrar na seção crítica em uma seção de entrada (**entry section**), executa e deve deixar a região crítica (**exit section**), continuando sua execução (**remainder section**)

- Estrutura geral do processo P_i

do {

entry section

critical section

exit section

remainder section

} while (true);

Algoritmo para o Processo P_i

do {

```
while (turn == j);
```

seção crítica

```
turn = j;
```

restante da seção

```
} while (true);
```

Solução para o Problema da Seção Crítica

1. **Exclusão Mútua** – se o processo P_i está executando em sua seção crítica, então nenhum processo pode estar executando em suas seções críticas.
2. **Progresso** – se nenhum processo está executando em sua seção crítica e existem alguns processos que desejam entrar em suas regiões críticas, então a seleção de processos que irão entrar na seção crítica em seguida não pode ser adiada indefinidamente
3. **Espera limitada (*bounded waiting*)** – um limite deve existir quanto ao número de vezes que outros processos são autorizados a entrar em suas seções críticas após um processo ter realizado um pedido para entrar em sua seção crítica e antes de sua requisição ser garantida
 - Assuma que cada processo executa em velocidade diferente de zero
 - Nenhuma afirmação pode ser feita com relação a **velocidade relativa** dos n processos

Tratamento de Seção Crítica em SO

- Duas abordagens, dependendo se o kernel é preemptivo ou não-preemptivo
- **Preemptivo** – permite a preempção de processos quando está executando em modo kernel
- **Não-preemptivo** – executa até sair do modo kernel, bloqueia ou libera a CPU voluntariamente
 - ▶ Essencialmente livre de condições de corrida em modo kernel

Solução de Peterson

- Boa descrição algorítmica para a solução do problema
- Solução para dois processos
- Assuma que as instruções de linguagem de máquina **load** e **store** são atômicas, de modo que não podem ser interrompidas
- Os dois processos compartilham duas variáveis
 - `int turn;`
 - `Boolean flag[2]`
- A variável **turn** indica quem irá entrar na seção crítica
- O vetor **flag** é usado para indicar se um processo está pronto para entrar a seção crítica. **flag[i] = true** implica que o processo P_i está pronto.

Algoritmo para o Processo P_i

do {

```
flag[i] = true;
```

```
turn = j;
```

```
while (flag[j] && turn == j); /*X*/
```

```
    critical section
```

```
flag[i] = false;
```

```
    remainder section
```

```
} while (true);
```

Solução de Peterson (Cont.)

- É possível provar que os três requisitos para tratamento de seção crítica (SC) são atendidos:
 1. Exclusão mútua é preservada
 P_i entra em SC somente:
se `flag[j] = false` ou `turn = i`
 2. Requisito com relação ao progresso é satisfeito
 3. Espera limitada é atendida
- **Problema:** *busy waiting* (veja o laço `/*X*/`).
- **Ex:** `peterson.c` e `nopeterson.c`

- Muitos sistemas oferecem suporte de hardware para implementação de código de SC
- Todas as soluções a seguir são baseadas na idéia de variável de intertravamento (**locking**)
 - Proteção de regiões críticas via locks
- Em uniprocessadores – poderia desabilitar interrupções
 - Código em execução em um dado momento poderia executar sem preempção
 - Geralmente muito ineficiente em sistemas multiprocessados
 - SOs usando esta solução não são amplamente escaláveis
- Máquinas modernas oferecem instruções de hardware atômicas
 - **Atômico** = não pode ser interrompido
 - Ou é um teste para uma palavra de memória e um ajuste de valor
 - Ou troca de conteúdos de duas palavras de memória

Solução para o problema de SC usando Locks

```
do {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
} while (TRUE);
```

Solução usando test_and_set()

- Variável Booleana compartilhada lock, inicializada em FALSE
- Solução:

```
do {  
    while (test_and_set(&lock)); /* não faça nada  
*/  
    /* seção crítica */  
    lock = false;  
    /* restante da seção */  
} while (true);
```

- Ex: testNset.c

Instrução test_and_set

Definição:

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```

1. Executado atomicamente
2. Retorna o valor original do parâmetro passado
3. Ajusta o valor do parâmetro passado para “TRUE”.

Instrução `compare_and_swap`

Definição:

```
int compare_and_swap(int *value, int expected,  
»                 int new_value) {  
    int temp = *value;  
  
    if (*value == expected)  
        *value = new_value;  
    return temp;  
}
```

1. Executada atomicamente
2. Retorna o valor original do parâmetro “value”
3. Ajusta a variável “value” para o valor do parâmetro passado “new_value”, mas apenas se “value == expected”. Assim, o *swap* ocorre somente sob essa condição

Solução usando compare_and_swap

- Inteiro compartilhado “lock” inicializado com 0
- Solução:

```
do {  
    while (compare_and_swap(&lock, 0, 1))  
        /* não fazer nada */  
    ;  
    /* seção crítica */  
    lock = 0;  
    /* restante da seção */  
} while (true);
```


- Embora os algoritmos apresentados satisfaçam a condição de exclusão mútua, eles não satisfazem o requisito de espera limitada.
- O próximo exemplo apresenta um algoritmo que usa `test_and_set()` e satisfaz os requisitos de SC.
 - Estruturas de dados comuns (inicializadas com **false**):

```
boolean waiting[n];
```

```
boolean lock;
```

Espera limitada com Exclusão Mútua usando test_and_set

```
do {  
    waiting[i] = true;  
    key = true;  
    while (waiting[i] && key)  
        key = test_and_set(&lock);  
    waiting[i] = false;  
    /* seção crítica */  
    j = (i + 1) % n;  
    while ((j != i) && !waiting[j])  
        j = (j + 1) % n;  
    if (j == i)  
        lock = false;  
    else  
        waiting[j] = false;  
    /* restante da seção */  
} while (true);
```

Espera limitada com Exclusão Mútua usando test_and_set

```
do {  
    waiting[i] = true;  
    while (waiting[i] && test_and_set(&lock));  
    waiting[i] = false;  
  
    /* critical section */  
  
    j = (i + 1) % n;  
    while ((j != i) && !waiting[j])  
        j = (j + 1) % n;  
    if (j == i)  
        lock = false;  
    else  
        waiting[j] = false;  
  
    /* remainder section */  
} while (true);
```

Observações

- **Exclusão mútua:** o processo P_i pode entrar na SC somente se $\text{waiting}[i] == \text{false}$ ou $\text{key} == \text{false}$.
 - key pode se tornar falso somente se o $\text{test_and_set}()$ é executado. O primeiro processo a executar o $\text{test_and_set}()$ encontrará $\text{key} == \text{false}$. Todos os outros devem esperar.
 - A variável $\text{waiting}[i]$ pode se tornar false somente se outro processo deixar sua região crítica; somente um $\text{waiting}[i]$ é ajustado para false, mantendo o requisito de exclusão mútua.
- **Progresso:** os argumentos apresentados para exclusão mútua também se aplicam aqui – um processo deixando sua SC ajusta $\text{lock} = \text{false}$ ou $\text{waiting}[j] = \text{false}$. Ambos permitem que um processo em espera entre em sua SC.

Observações (cont)

- **Espera limitada:** quando um processo deixa sua SC, ele verifica o vetor `waiting[]` em ordem cíclica ($i + 1, i + 2, \dots, n - 1, 0, \dots, i - 1$). Ele designa o primeiro processo nesta ordem que está na seção de entrada (`waiting[j] == true`) como o próximo a entrar na SC.
 - Cada processo esperando para entrar em sua SC fará isso dentro de $n-1$ turnos.
- Detalhes de implementação de `test_and_set()` e `compare_and_swap()` podem ser encontrados em livros de arquitetura de computadores.
- **Ex:** `testNset-v2.c`

- Soluções prévias são complicadas e geralmente inacessíveis a programadores de aplicação
- Projetistas de SO criam ferramentas de software para resolver o problema da seção crítica
- As mais simples são os locks mutex
 - No contexto de threads, é um objeto de propriedade desta.
 - Permite que apenas uma única thread acesse o recurso

Mutex Locks (cont.)

- Protege uma seção crítica através da aquisição (**acquire()**) do *lock* e sua liberação posterior (**release()**)
 - Variável booleana indica se o *lock* está disponível ou não
- As chamadas **acquire()** e **release()** devem ser atômicas
 - Geralmente implementadas via instruções de hardware atômicas
- Esta solução requer espera ocupada (**busy waiting**)
 - Este intertravamento também é chamado de **spinlock**

acquire() e release()

```
□ acquire() {  
    while (!available)  
        ; /* busy wait */  
    available = false;;  
}  
  
□ release() {  
    available = true;  
}  
  
□ do {  
    acquire (lock)  
    critical section  
    release (lock)  
    remainder section  
} while (true);
```




Ex.:

- `mutex_ex.c`